



INSTITUTO TECNOLÓGICO SUPERIOR DE LERDO.



- **Alumno:**

Rodriguez Rosales Diego Guillermo.

- **Num. Control:**

#222310306.

- **Docente:**

Jesus Salas Marin.

- **Materia:**

Desarrollo de Aplicaciones para Dispositivos Móviles.

- **Clave:**

4Y8A.

- **Actividad:**

Act 4.1 Creación de un Mini Reproductor de Música en Flutter.

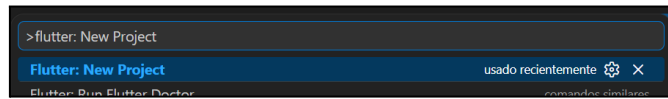
19-Mayo-2026.



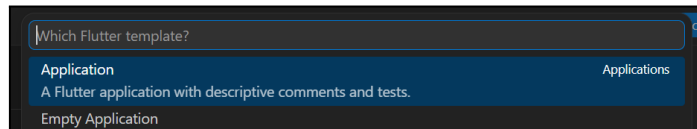


Capítulo 1. Creación del proyecto.

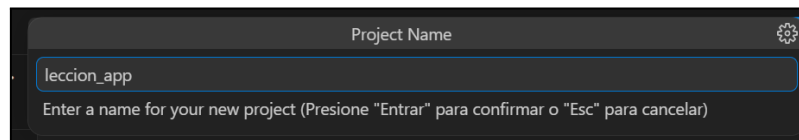
1. Iniciamos usando el comando Ctrl+Shift+P para abrir el buscador y escribimos “flutter new” y seleccionamos “Flutter: New Project”.



2. Y seleccionamos la opción de Application y elegimos la carpeta en donde se creará el proyecto (es donde deseemos donde se guardará la carpeta).

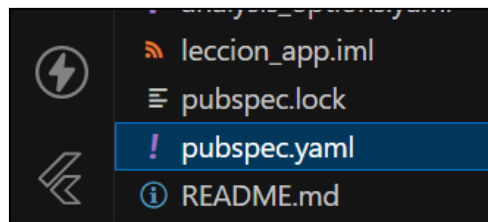


3. Para seguir, asignamos el nombre del proyecto, en este caso será “mini_repr_app”.





4. Iremos al archivo “pubspec.yaml”.



- Y en el archivo reemplazamos el contenido con el siguiente código.

```
1  name: app2
2  description: "A new Flutter project."
3
4  publish_to: 'none' # Remove this line if you wish to publish to pub.dev
5
6  version: 1.0.0+1
7
8  environment:
9    sdk: ^3.7.0
10
11 # The following section is specific to Flutter.
12 dependencies:
13   flutter:
14     sdk: flutter
15   just_audio: ^0.10.2
16   rxdart: ^0.28.0
17
18 cupertino_icons: ^1.0.8
19
20 dev_dependencies:
21   flutter_test:
22     sdk: flutter
23
24   flutter_lints: ^5.0.0
25
26 flutter:
```



Capítulo 2. Creación de archivos.

- main.dart:
 - Este es el punto de entrada de la aplicación.
 - Qué hace.
 - Inicia Flutter.
 - Ejecuta la app.
 - Muestra la pantalla principal.

```
1 import 'package:flutter/material.dart';
2
3 import 'music_player_screen.dart';
4
5 void main() {
6   runApp(const MusicPlayerApp());
7 }
8
9 class MusicPlayerApp extends StatelessWidget{
10   const MusicPlayerApp({super.key});
11
12   @override
13   Widget build(BuildContext context) {
14     return MaterialApp(
15       debugShowCheckedModeBanner: false,
16       title: 'Music Player',
17       home: const MusicPlayerScreen(),
18     );
19   }
20 }
```



- position_data.dart:
 - Este archivo guarda información de la canción.

```
1 class PositionData {  
2     final Duration position;  
3     final Duration bufferedPosition;  
4     final Duration duration;  
5  
6     PositionData(  
7         this.position, this.bufferedPosition, this.duration);  
8 }
```



- seek_bar.dart:
 - Este archivo crea la barra de duración de la canción.
 - Qué hace.
 - Muestra progreso
 - Permite adelantar canción
 - Muestra tiempo actual
 - Muestra duración total

```
1 import 'package:flutter/material.dart';
2
3 class SeekBar extends StatelessWidget {
4   final Duration duration;
5   final Duration position;
6   final Duration bufferedPosition;
7   final ValueChanged<Duration>? onChangeEnd;
8
9   SeekBar({
10     required this.duration,
11     required this.position,
12     required this.bufferedPosition,
13     required this.onChangeEnd,
14   });
15
16   @override
17   Widget build(BuildContext context) {
18     return Column(
19       children: [
20         Slider(
21           activeColor: Colors.yellow,
22           inactiveColor: Colors.grey,
23
24           min: 0.0,
25           max: duration.inMilliseconds.toDouble(),
26
27           value: position.inMilliseconds
28             .clamp(0, duration.inMilliseconds)
29             .toDouble(),
30
31           onChanged: (value) {
32             onChangeEnd?.call(
33               Duration(milliseconds: value.round()),
34             );
35           },
36         ),
37         Row(
38           mainAxisAlignment: MainAxisAlignment.spaceBetween,
39           children: [
40             Text(
41               _formatDuration(position),
42               style: const TextStyle(color: Colors.white),
43             ),
44             Text(
45               _formatDuration(duration),
46               style: const TextStyle(color: Colors.white),
47             ),
48           ],
49         ),
50       ],
51     );
52   }
53
54   String _formatDuration(Duration duration) {
55     String twoDigits(int n) => n.toString().padLeft(2, '0');
56     final minutes = twoDigits(duration.inMinutes.remainder(60));
57     final seconds = twoDigits(duration.inSeconds.remainder(60));
58     return '$minutes:$seconds';
59   }
60 }
61
62
```



- music_player_screen.dart:
 - Este es el archivo MÁS importante.

Aquí está toda la interfaz del reproductor.

- Qué hace.
- Muestra imagen
- Muestra nombre canción
- Reproduce audio
- Pausa audio
- Muestra barra de progreso
- Controla botones

```
import 'package:flutter/material.dart';
import 'package:just_audio/just_audio.dart';
import 'package:rxdart/rxdart.dart';

import 'position_data.dart';
import 'seek_bar.dart';

class MusicPlayerScreen extends StatefulWidget {
  const MusicPlayerScreen({super.key});

  @override
  State<MusicPlayerScreen> createState() =>
    _MusicPlayerScreenState();
}

class _MusicPlayerScreenState
  extends State<MusicPlayerScreen> {
  late AudioPlayer _audioPlayer;

  bool isPlaying = false;

  String currentTrack = 'The Adults Are Talking';
  String artist = 'The Strokes';

  @override
  void initState() {
    super.initState();

    _audioPlayer = AudioPlayer();
```



```
        _loadAudio();
    }

    Future<void> _loadAudio() async {

        try {

            await _audioPlayer.setAsset(

                'assets/The Adults Are Talking.m4a',

            );

        } catch (e) {

            debugPrint(

                'Error loading audio: $e',

            );

        }

    }

    @override
    void dispose() {

        _audioPlayer.dispose();

        super.dispose();

    }

    Stream<PositionData> get _positionDataStream =>

        Rx.combineLatest3<

            Duration,

            Duration,

            Duration?,

            PositionData

        >(

            _audioPlayer.positionStream,

            _audioPlayer.bufferedPositionStream,

            _audioPlayer.durationStream,

            (

                position,

                bufferedPosition,

                duration,

            ) =>
```




```
        PositionData(  
            position,  
            bufferedPosition,  
            duration ?? Duration.zero,  
        ),  
    );  
  
void _playPause() async {  
    if (isPlaying) {  
        await _audioPlayer.pause();  
    } else {  
        await _audioPlayer.play();  
    }  
  
    setState(() {  
        isPlaying = !isPlaying;  
    });  
}  
  
void _nextTrack() {  
    // Próxima canción  
}  
  
void _previousTrack() {  
    // Canción anterior  
}  
  
@override  
Widget build(BuildContext context) {  
    return Scaffold(  
        backgroundColor:  
            const Color(0xFF2C2C54),  
  
        appBar: AppBar(  
            backgroundColor:  
                const Color(0xFF2C2C54),
```



```
elevation: 0,

centerTitle: true,

title: const Text(

  'Hearme',

  style: TextStyle(

    color: Colors.white,

    fontWeight: FontWeight.bold,

  ),

),

),

body: Padding(

  padding:

    const EdgeInsets.all(20.0),

  child: Column(

    mainAxisAlignment:

      MainAxisAlignment.center,

    children: [

      // Imagen

      Container(

        width: 250,

        height: 250,

        decoration:

          const BoxDecoration(

            shape: BoxShape.circle,

            image: DecorationImage(

              image: AssetImage(

                'assets/artist.png',

              ),

            ),
```



```
        fit: BoxFit.cover,

      ),

    ),

  ),

  const SizedBox(height: 30),

  // Canción

  Text(

    currentTrack,

    textAlign:

      TextAlign.center,

    style: const TextStyle(

      fontSize: 24,

      fontWeight:

        FontWeight.bold,

      color: Colors.white,

    ),

  ),

  const SizedBox(height: 10),

  // Artista

  Text(

    artist,

    style: const TextStyle(

      fontSize: 18,

      color: Colors.white70,

    ),

  ),

  const SizedBox(height: 30),

  // Barra de progreso
```



```
StreamBuilder<PositionData>(  
  stream:  
    _positionDataStream,  
  
  builder:  
    (context, snapshot) {  
      final positionData =  
        snapshot.data;  
  
      return SeekBar(  
        duration:  
          positionData  
            ?.duration ??  
            Duration.zero,  
  
        position:  
          positionData  
            ?.position ??  
            Duration.zero,  
  
        bufferedPosition:  
          positionData  
            ?.bufferedPosition ??  
            Duration.zero,  
  
        onChangeEnd:  
          (newPosition) {  
            _audioPlayer.seek(  
              newPosition,  
            );  
          },  
        );  
      ),  
    ),  
  
  const SizedBox(height: 30),  
)
```



```
// Botones

Row(

  mainAxisAlignment:

    MainAxisAlignment

      .center,

  children: [

    // Previous

    IconButton(

      iconSize: 48,

      icon: const Icon(

        Icons.skip_previous,

        color:

          Colors.white,

      ),

      onPressed:

        _previousTrack,

    ),

    const SizedBox(

      width: 20,

    ),

    // Play Pause

    IconButton(

      iconSize: 72,

      icon: Icon(

        isPlaying

          ? Icons

            .pause_circle_filled

          : Icons

            .play_circle_fill,

        color:
```



```
        Colors.white,

      ),

      onPressed:

        _playPause,

      ),

      const SizedBox(

        width: 20,

      ),

      // Next

      IconButton(

        iconSize: 48,

        icon: const Icon(

          Icons.skip_next,

          color:

            Colors.white,

        ),

        onPressed:

          _nextTrack,

      ),

    ],

  ),

],

),

),

);

}
```

- Resultado:

https://drive.google.com/file/d/1tF5lFBF6Jk5mTDW5EC9Z3Z2Ukzff_Qjn/view?usp=sharing



Preguntas de Reflexión.

1. ¿Qué retos encontraste al seguir las instrucciones del video?

Cómo acomodar las clases y tener una estructura clara para corregir los errores.

2. ¿Qué diferencia notaste entre una aplicación que simula una interfaz y una que realmente reproduce audio?

Está más limitado cuando se reproducen los audios.

3. ¿Qué widgets consideras más importantes o esenciales para construir una interfaz de un reproductor en Flutter?

Los controles de reproducción de música.